

Engineering Challenges

#2 Intent Qualification (ML Engineer Intern)

Applicant: Scutaru Stefan-Daniel

1.Introduction

I chose this problem from among the available options because it seemed to me to be the most interesting and challenging. The first question I asked myself was what “match” really means in this context: not textual similarity, but fulfilling the intent behind the query. That guided all my design decisions.

To run the application locally:

1. Clone the repository from GitHub.
2. Install the dependencies: `pip install -r requirements.txt`
3. Create a .env file in the root directory and add the API key:
`ANTHROPIC_API_KEY=ANTHROPIC_API_KEY= {Sent via Email}`
4. Start the application: `streamlit run app.py`

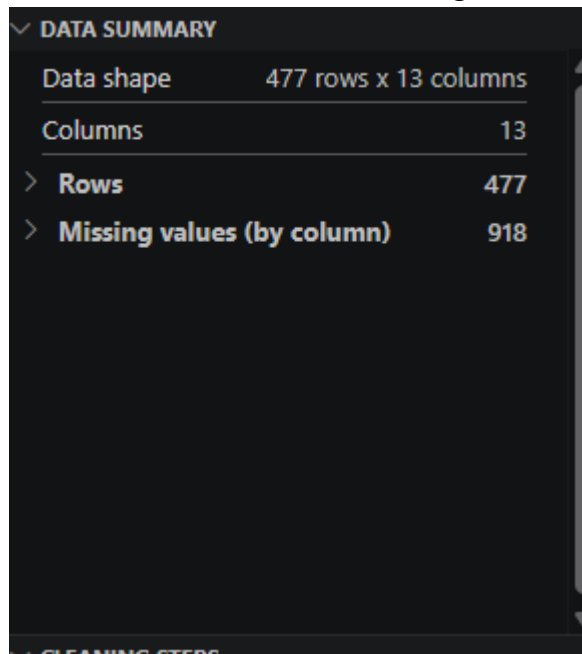
The first time you run it, the embeddings model downloads automatically. After that, everything runs locally.

2.The first interaction with data

The first thing I did was open the `companies.jsonl` file and figure out what I was working with. Coming from a relational database background, the format took me by surprise. There is no fixed schema; each line is a separate JSON object, and the fields are not consistent across records. Some companies have a “revenue” field, others do not. Some have “secondary_naics” as a list, while others do not have that field at all.

The results of the exploration were clear: 449 companies, with approximately 20% missing values in demographic fields such as number of employees and revenue. The semantic fields (description, industry, products, and target markets) had zero missing values, which confirmed

to me that I could build the ranking based on them without worrying about incomplete data.



A screenshot of a data summary panel with a dark background. The panel is titled 'DATA SUMMARY' with a downward arrow. It contains the following information:

Data shape	477 rows x 13 columns
Columns	13
> Rows	477
> Missing values (by column)	918

Below the summary, there is a section titled 'CLEANING STEPS' which is partially visible.

I also checked the business model field, which contained 18 distinct values. Instead of hard-coding them into the prompt, I decided to dynamically extract them from the dataset on each run, so that the list would update automatically if new companies appeared.

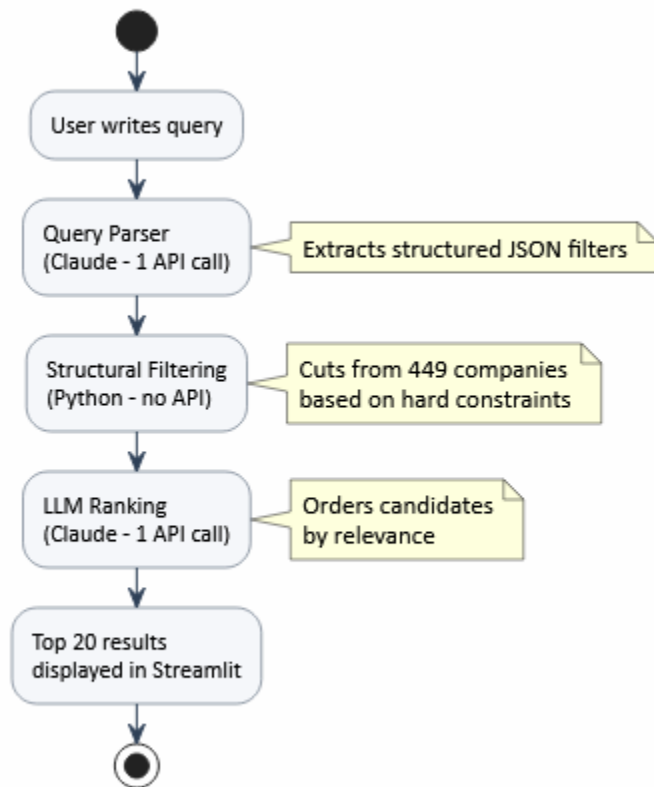
3.Problem Definition and Architecture

Before writing any code, I spent some time thinking about what “match” actually means in this context. The prompt gave a concrete example: a search for “Companies supplying packaging for cosmetics brands” might return cosmetics brands instead of suppliers, because both include the same words. Textual similarity doesn’t capture the role a company plays in an ecosystem.

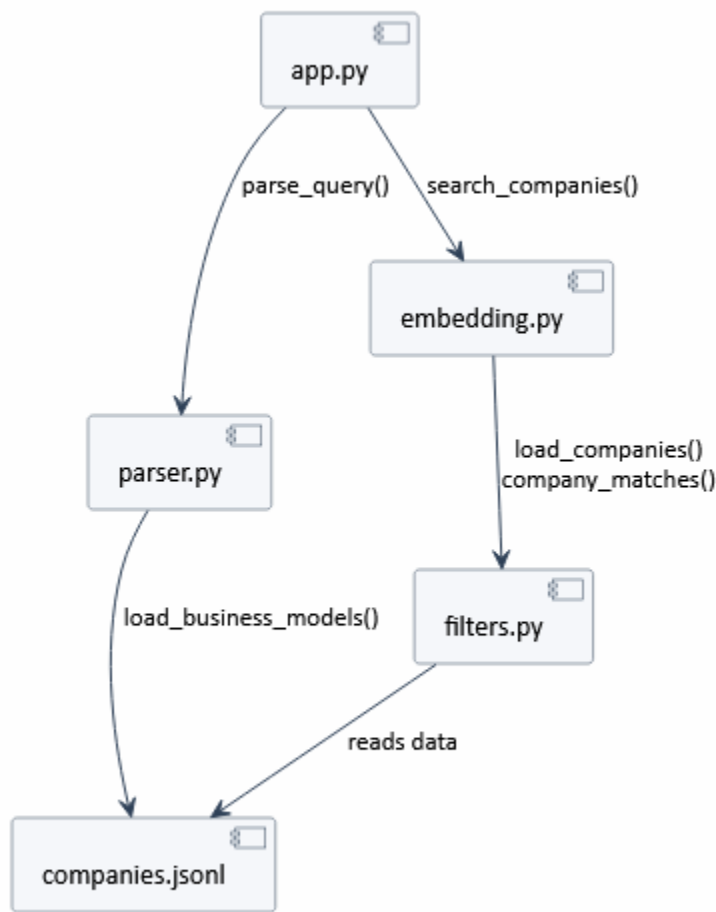
The prompt presented two baseline approaches. I tried to implement Baseline B, which ranks results based on embedding similarity, but it didn’t work out. The concept was hard to understand, and implementing it correctly turned out to be more complicated than I expected. I decided to go with a simpler and more straightforward solution: structural filtering followed by ranking via an LLM. A call parses the query and extracts the explicit constraints; filters are applied to the dataset; and the remaining companies are sent to Claude, which ranks them by relevance.

- LLM Parser: converts the query into structured filters (JSON), one API call per query
- Structured Filtering: applies the filters to the dataset using Python, instantly and at no cost

- LLM Ranking: sends the filtered candidates to Claude for semantic ranking, one API call



Compared to an embedding-based approach, this solution is simpler to implement, easier to understand, and produces more accurate rankings for complex queries that require reasoning. The additional cost compared to embedding similarity is justified by the superior quality of the results.



3.1. Parser file

The first component I built was the parser: a single call to Claude that converts the query into structured JSON filters.

Writing the prompt was harder than I expected and went through several iterations. The main problem was that the model interpreted numerical operators ambiguously; for example, “more than 100” could mean 100 or 101. I explicitly defined the rules and ran the prompt through Claude, asking it to identify where it was unclear, which led to more consistent output.

During testing, I added new fields as I encountered queries that the initial version couldn’t handle: “country” was converted into an array of ISO codes to support regions like “Scandinavia” or “Europe,” “company_type” to distinguish manufacturers from distributors, and fields for the year of founding for queries asking for recent startups.

3.2. Filters file

The filters file applies the JSON produced by the parser directly to the company list. Each company is checked against every active filter, and only companies that satisfy all constraints pass through.

Country filtering now accepts an array of ISO codes instead of a single value, which allows region-based queries to work correctly. When the parser returns a list of codes for "Scandinavia", the filter checks whether the company's country code appears anywhere in that list.

The `in_range` function handles all numeric constraints uniformly: minimum, maximum, and exact values for employee count, revenue, and year founded. If a company is missing a field that a filter targets, it is excluded. This is a conservative approach that prioritizes precision over recall.

The `company_type` filter checks the NAICS label directly: if the query asks for manufacturers, the label must contain "manufactur"; for distributors, it must contain "wholesal" or "distribut".

The industry filter is intentionally disabled at this stage. NAICS labels are too technical and specific to match reliably against the general terms users write in queries. That mapping is handled by the LLM ranker instead.

3.3. Embedding file

After structural filtering reduces the candidate pool, the remaining companies are sent to Claude in a single API call for ranking. For each company, a short text is built from the name and the first 200 characters of the description. This list, together with the original query, is sent to the model, which returns a JSON array of company indices ordered by relevance.

This approach replaces embedding similarity with direct LLM reasoning. The model understands that a packaging supplier and a cosmetics brand play different roles even when their descriptions share similar vocabulary, which was the core failure of Baseline B.

The trade-off is cost: unlike embeddings that run locally at no cost, this API call adds expense per query. For 449 companies with aggressive structural filtering beforehand, the number of companies sent to ranking is small, so the cost remains acceptable.

The file is named `embedding.py` for historical reasons, as it originally contained the embedding-based implementation before it was replaced.

4. Query results

Query 1: "Logistic companies in Romania"

what company are you looking for?

Logistic companies in Romania

Search

Filters found:

```
{
  "country": [
    0: "RO"
  ]
  "industry": "logistics"
  "min_employees": NULL
  "max_employees": NULL
  "exact_employees": NULL
  "min_revenue": NULL
  "max_revenue": NULL
  "exact_revenue": NULL
  "is_public": NULL
  "business_model": [
    0: "Logistics/Transportation"
  ]
  "company_type": NULL
  "min_year_founded": NULL
  "max_year_founded": NULL
  "exact_year_founded": NULL
}
```

Name	Website	Country	Industry
Brasov Industrial Portfolio	brasov-industrialportfolio.ro	ro	General Warehousing and Storage
Compania Națională de Căi Ferate	cfr.ro	ro	Support Activities for Rail Transportation

Query 2: "Public software companies with more than 1,000 employees"

what company are you looking for?

Public software companies with more than 1,000 employees

Search

Filters found:

```
{
  "country": NULL
  "industry": "software"
  "min_employees": 1001
  "max_employees": NULL
  "exact_employees": NULL
  "min_revenue": NULL
  "max_revenue": NULL
  "exact_revenue": NULL
  "is_public": true
  "business_model": NULL
  "company_type": NULL
  "min_year_founded": NULL
  "max_year_founded": NULL
  "exact_year_founded": NULL
}
```

Name	Website	Country	Industry
Tata Consultancy Service	tcs.com	in	Computer Systems Design Services
Wipro	wipro.com	in	Computer Systems Design Services
CGI	cgi.com	ca	Computer Systems Design Services
EPAM	epam.com	us	Custom Computer Programming Services
NTT DATA	nttdata.com	jp	Computer Systems Design Services
Fujitsu	fujitsu.com	jp	Computer Systems Design Services

Query 3: "Food and beverage manufacturers in France"

Food and beverage manufacturers in France

Search

Filters found:

{

"country" : [

0 : "FR"

]

"industry" : "food and beverage"

"min_employees" : NULL

"max_employees" : NULL

"exact_employees" : NULL

"min_revenue" : NULL

"max_revenue" : NULL

"exact_revenue" : NULL

"is_public" : NULL

"business_model" : [

0 : "Manufacturing"

]

"company_type" : "manufacturer"

"min_year_founded" : NULL

"max_year_founded" : NULL

"exact_year_founded" : NULL

}

Name	Website	Country	Industry	Empl
Famille Pouille	pouille.eu	fr	All Other Miscellaneous Food Manufacturing	
La Limonaderie de Paris	lalimonaderiedeparis.com	fr	Soft Drink Manufacturing	
Auvergnat Cola	auvergnatcola.com	fr	Soft Drink Manufacturing	
Geyer	None	fr	Soft Drink Manufacturing	

Query 4: "Companies that could supply packaging materials for a direct-to-consumer cosmetics brand"

Companies that could supply packaging materials for a direct-to-consumer cosmetics brand

Search

Filters found:

{

"country" : NULL

"industry" : "packaging materials"

"min_employees" : NULL

"max_employees" : NULL

"exact_employees" : NULL

"min_revenue" : NULL

"max_revenue" : NULL

"exact_revenue" : NULL

"is_public" : NULL

"business_model" : [

0 : "Business-to-Business"

1 : "Manufacturing"

2 : "Wholesale"

"company_type" : "manufacturer"

"min_year_founded" : NULL

"max_year_founded" : NULL

"exact_year_founded" : NULL

}

Name	Website	Country	Industry	Empl
BAOLILONG GLASS PACKAGING	baolilongglasspackaging.com	cn	Glass Container Manufacturing	
YONWOO	yonwookorea.com	kr	Plastics Bottle Manufacturing	
Wenjing Packaging	wenjingpack.com	cn	Plastics Bottle Manufacturing	
Shanghai Bochen Cosmetics	cn.cosmeticbottles.com	cn	Plastics Bottle Manufacturing	

Query 5: "Construction companies in the United States with revenue over \$50 million"

what company are you looking for?

Construction companies in the United States with revenue over \$50 million

Search

Filters found:

```
{
  "country": [
    0 : "US"
  ]
  "industry": "construction"
  "min_employees": NULL
  "max_employees": NULL
  "exact_employees": NULL
  "min_revenue": 50000001
  "max_revenue": NULL
  "exact_revenue": NULL
  "is_public": NULL
  "business_model": NULL
  "company_type": NULL
  "min_year_founded": NULL
  "max_year_founded": NULL
  "exact_year_founded": NULL
}
```

Name	Website	Country	Industry
Kiewit	kiewit.com	us	Other Heavy and Civil Engineering Construction
Bechtel	bechtel.com	us	Other Heavy and Civil Engineering Construction
Fluor	fluor.com	us	Other Heavy and Civil Engineering Construction
EMCOR Group	emcorgroup.com	us	Electrical Contractors and Other Wiring Installation Contractors

Query 6: "Pharmaceutical companies in Switzerland"

what company are you looking for?

Pharmaceutical companies in Switzerland

Search

Filters found:

```
{
  "country": [
    0 : "CH"
  ]
  "industry": "pharmaceutical"
  "min_employees": NULL
  "max_employees": NULL
  "exact_employees": NULL
  "min_revenue": NULL
  "max_revenue": NULL
  "exact_revenue": NULL
  "is_public": NULL
  "business_model": NULL
  "company_type": NULL
  "min_year_founded": NULL
  "max_year_founded": NULL
  "exact_year_founded": NULL
}
```

Name	Website	Country	Industry	Employ
Novartis	novartis.com	ch	Pharmaceutical Preparation Manufacturing	75
Lonza	lonza.com	ch	Pharmaceutical Preparation Manufacturing	18
Octapharma	octapharma.com	ch	Biological Product (except Diagnostic) Manufacturing	11
Ferring	ferring.com	ch	Pharmaceutical Preparation Manufacturing	7

Query 7: "B2B SaaS companies providing HR solutions in Europe"

what company are you looking for?

B2B SaaS companies providing HR solutions in Europe

Search

Filters found:

{

"country" : [...]

"industry" : "hr solutions"

"min_employees" : NULL

"max_employees" : NULL

"exact_employees" : NULL

"min_revenue" : NULL

"max_revenue" : NULL

"exact_revenue" : NULL

"is_public" : NULL

"business_model" : [

0 : "Business-to-Business"

1 : "Software-as-a-Service"

]

"company_type" : NULL

"min_year_founded" : NULL

"max_year_founded" : NULL

"exact_year_founded" : NULL

}

Name	Website	Country	Industry	Employees	Revenue	Public	Business
Personio	None	de	Software Publishers	None	None	☐	Business-
Ciphr	ciphr.com	gb	Software Publishers	230	46103753	☐	Service P
Zellis	zellis.com	gb	Software Publishers	1400	307376497	☐	Service P
Factorial	factorialhr.de	es	Software Publishers	1	53626	☐	Business-

Query 8: "Clean energy startups founded after 2018 with fewer than 200 employees"

what company are you looking for?

Clean energy startups founded after 2018 with fewer than 200 employees

Search

Filters found:

{

"country" : NULL

"industry" : "clean energy"

"min_employees" : NULL

"max_employees" : 199

"exact_employees" : NULL

"min_revenue" : NULL

"max_revenue" : NULL

"exact_revenue" : NULL

"is_public" : NULL

"business_model" : NULL

"company_type" : NULL

"min_year_founded" : 2019

"max_year_founded" : NULL

"exact_year_founded" : NULL

}

Name	Website	Country	Industry
Norhybrid Renewables	norhybrid.com	no	Turbine and Turbine Generator Set Units Manufacturing
Norhybrid Renewables	norhybrid.com	no	Turbine and Turbine Generator Set Units Manufacturing
SideWind	sidewind.is	is	Turbine and Turbine Generator Set Units Manufacturing
Next Fabrication	nextfabrication.com	dk	Fabricated Structural Metal Manufacturing
Next Fabrication	nextfabrication.com	dk	Fabricated Structural Metal Manufacturing
E-Lyte	e-lyte.de	de	All Other Miscellaneous Chemical Product and Preparation

Query 9: "Fast-growing fintech companies competing with traditional banks in Europe"

Fast-growing fintech companies competing with traditional banks in Europe

Search

Filters found:

{

▶ "country" : [...]

"industry" : "fintech"

"min_employees" : NULL

"max_employees" : NULL

"exact_employees" : NULL

"min_revenue" : NULL

"max_revenue" : NULL

"exact_revenue" : NULL

"is_public" : NULL

▼ "business_model" : [

0 : "Business-to-Consumer"

1 : "Business-to-Business"

]

"company_type" : NULL

"min_year_founded" : NULL

"max_year_founded" : NULL

"exact_year_founded" : NULL

}

Name	Website	Country	Industry
Aircash	aircash.eu	hr	Financial Transactions Processing, Reserve, and C
Phin	phintech.de	de	Financial Transactions Processing, Reserve, and C
Folio	folio.no	no	Financial Transactions Processing, Reserve, and C
European Pay	european-pay.fr	fr	Financial Transactions Processing, Reserve, and C

5. Where it works well and where it does not

The system works well on queries with clear structural constraints. A query like "public software companies with more than 1,000 employees" is essentially a SQL filter: the parser extracts the fields correctly and filtering returns exactly the right candidates. The LLM ranker confirms and refines the order.

Region-based queries also work well. A query mentioning "Scandinavia" automatically receives the corresponding list of ISO codes from the parser, and filtering works correctly without any changes to the filtering logic.

LLM ranking improves results significantly on complex queries compared to embedding similarity. The model understands the difference between a supplier and a client, between a company that produces and one that distributes.

Where the system can go wrong is when structural filters do not cut aggressively enough and the LLM ranker receives a long list. With many companies in the prompt, the model may make ordering errors or exceed the token limit. Descriptions truncated to 200 characters may also not be sufficient to differentiate similar companies.

A clear downside of the application is that it makes two LLM API calls per query. On simple queries where filters cut aggressively, the second call is fast. On vague queries without clear filters, many candidates remain and the ranking call becomes slow and more expensive.

Missing data also introduces a silent problem: a company without an employee count is excluded by any query that specifies a minimum, even if it would otherwise qualify. This conservative approach trades recall for precision.

6. Scaling

The current implementation loads all companies into memory and filters them in a loop. At 449 companies this works well. At 100,000 companies, two things would break down.

The in-memory company list would need to be replaced with a database: PostgreSQL with indices on country code, employee count, revenue, and year founded. Filters would become SQL WHERE clauses, far more efficient than iterating a Python list.

LLM ranking has a critical limitation at scale: it is not possible to send thousands of companies in a single prompt. At 100,000 companies, structural filtering must cut aggressively before ranking. A hybrid approach would use embedding similarity for a fast pre-filter to top 50 or 100, with the LLM ranking only those.

At large scale, many users would submit similar queries. A cache would store results for identical or very similar queries, significantly reducing LLM API calls.

7. Failure modes and what I would monitor in production

The most dangerous failure mode is when structural filters do not cut enough and the LLM ranker receives a list that is too long. With hundreds of companies in the prompt, the model can make ordering errors or exceed the token limit. I would monitor the number of companies sent to ranking and set a hard maximum.

The parser can extract incorrect filters from ambiguous queries: a country not mentioned, an overly aggressive revenue filter, or a wrong expansion of a geographic region. In production I would log all extracted JSON objects and monitor for filters that exclude more than 90% of the dataset, filters with all fields null, and country codes that do not exist in the dataset.

Descriptions truncated to 200 characters for ranking may not capture relevant information about a company. A company with a long and relevant description at the end may be ranked incorrectly against one with key information at the start.

Companies with missing data are conservatively excluded by structural filters. I would monitor how many companies are excluded due to missing data versus how many fail the filters on actual values, to understand whether relevant candidates are being lost.

8. Final notes

The part I found most interesting was the prompt design for the parser. It is surprisingly difficult to write instructions that are both precise enough to produce consistent structured output and flexible enough to handle the full range of query types. I refined the prompt more than any other part of the project.

The decision to replace embedding similarity with direct LLM ranking simplified the code significantly and improved result quality on complex queries. The cost trade-off is real but justified for the current use case.

If I were to continue the project, I would explore a hybrid approach for scalability: embedding similarity for a fast pre-filter to top 50, followed by LLM ranking on those. This would preserve the current ranking quality at a controlled cost and larger scale.